

Software Requirements Specification

For

AI-Based Tourism Pressure & Ecosystem Stress Management Dashboard

March 2025

Prepared by

Specialization	SAP ID	Name
Data Science	500123465	Sangam Khanna
AIML	500123471	Manan Nasa



Department of Informatics
School of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN – 248007, Uttarakhand

REVISION HISTORY

Date	Change	Reason for Changes	Mentor Signature
March 2025	Initial Draft	Project Inception	

1 INTRODUCTION

1.1 Purpose of the Project

The primary purpose of this project is to develop an AI-powered Decision Support Dashboard for monitoring, analysing, and predicting tourism pressure in the Char Dham pilgrimage region of Uttarakhand, India. The Char Dham circuit — comprising the four sacred shrines of Kedarnath, Badrinath, Gangotri, and Yamunotri — attracts millions of pilgrims annually and is located in ecologically sensitive sub-Himalayan terrain. Rapid increases in visitor footfall, compounded by climate variability, impose significant stress on fragile mountain ecosystems, local infrastructure, and public safety systems.

Existing tourism management approaches in the region are predominantly reactive, relying on post-season analysis rather than proactive forecasting. The absence of an integrated, data-driven decision support tool means that environmental thresholds are routinely breached before corrective action is taken. This project addresses that critical gap by designing and implementing an end-to-end software system that consolidates historical tourist footfall data and meteorological data, applies statistical and machine learning techniques to extract actionable intelligence, and presents outputs through an interactive web-based dashboard.

The motivation behind this project is grounded in three imperatives: (i) ecological preservation — providing timely early warnings when ecosystem carrying capacities approach critical thresholds; (ii) administrative efficiency — equipping tourism authorities and policy makers with predictive insights to plan regulated entry, infrastructure deployment, and emergency preparedness; and (iii) academic contribution — demonstrating the viability of combining Machine Learning, Explainable AI, and geospatial visualisation to solve a real-world socio-environmental challenge.

1.2 Target Beneficiary

The system is designed to serve four primary user groups:

- Tourism Authorities (e.g., Uttarakhand Tourism Development Board): These agencies require predictive footfall data and stress-level alerts to regulate pilgrim access, coordinate logistics, and enforce carrying-capacity limits in compliance with the National Green Tribunal directives.
- Environmental Monitoring Agencies (e.g., UEPPCB, Forest Survey of India): These stakeholders need quantitative Ecosystem Stress Index (ESI) metrics and trend visualisations to assess anthropogenic impact on biodiversity, soil, water, and vegetative cover in the Char Dham zone.

- Research Scholars and Academic Institutions: Researchers studying climate–tourism interactions, Himalayan ecology, or sustainable development benefit from the system's analytical modules, correlation matrices, SHAP explainability outputs, and open dataset integration framework.
- Policy Makers and Government Administrators: District collectors, state secretaries, and planning commissions require high-level decision support summaries, scenario-based forecasts, and AI-generated operational recommendations to inform infrastructure investment and regulatory policy.

1.3 Project Scope

The scope of this project encompasses the complete software lifecycle from data ingestion through analytical modelling to interactive visualisation and alert generation. The system operates on two primary datasets — a Tourist Footfall Dataset and a Climate Dataset — both covering the four Char Dham shrines across multiple years and months. The merged dataset provides the basis for all downstream analytical operations.

The system delivers the following major functional capabilities:

1. Data Integration and Pre-processing: Automated merging of footfall and climate datasets, temporal feature engineering, and missing-value handling using Pandas and NumPy.
2. Ecosystem Stress Index (ESI) Computation: A composite, weighted metric that quantifies environmental stress at each shrine on a normalised scale, based on pilgrim density relative to carrying capacity, temperature deviation, and rainfall deviation.
3. Exploratory Data Analysis (EDA): Interactive charts, seasonality heatmaps, scatter plots, and correlation matrices rendered using Plotly within the Streamlit interface.
4. Machine Learning Prediction — Random Forest Regression: Supervised learning model predicting monthly pilgrim counts using meteorological features (month, average temperature, rainfall).
5. Time-Series Forecasting — ARIMA Model: Statistical time-series model producing multi-step-ahead forecasts of pilgrim counts using historical temporal data.
6. Model Evaluation: Quantitative assessment using RMSE, MAE, and R^2 score, complemented by feature importance and residual error distribution visualisations.
7. AI Early Warning Alert System: Automated classification of ESI into three alert levels (Green/Yellow/Red) with shrine-level status indicators.
8. AI Recommendation Engine: Context-sensitive operational recommendations generated dynamically based on ESI thresholds and predicted pilgrim counts.
9. Geospatial Monitoring Map: Interactive choropleth or marker map of Char Dham shrines displaying real-time stress indicators and pilgrim statistics.

10. Explainable AI (SHAP): SHAP value computation and visualisation to promote transparency and interpretability of Random Forest model predictions.

The system is implemented entirely in Python, deployed as a Streamlit web application, and requires no dedicated server infrastructure beyond a standard Python runtime environment. Mobile application development, cloud-native deployment, and live API integrations with government databases are explicitly out of scope for the current version.

1.4 References

Ref #	Document / Source	URL / Citation
[1]	Uttarakhand Tourism Development Board – Annual Pilgrim Statistics	https://uttarakhandtourism.gov.in
[2]	Sacred Yatra – Char Dham Visitor Records	https://badrinath-kedarnath.gov.in
[3]	India Meteorological Department – Climate Data Portal	https://imd pune.gov.in
[4]	National Green Tribunal Order on Char Dham Carrying Capacity, 2019	NGT Original Application No. 264/2019
[5]	Scikit-learn Documentation – Random Forest Regressor	https://scikit-learn.org/stable
[6]	Statsmodels Documentation – ARIMA	https://www.statsmodels.org
[7]	SHAP: A Unified Approach to Interpreting Model Predictions (Lundberg & Lee, 2017)	https://shap.readthedocs.io
[8]	Streamlit Framework Documentation	https://docs.streamlit.io
[9]	Plotly Python Graphing Library Documentation	https://plotly.com/python
[10]	IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications	IEEE Computer Society, 1998

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

The system implements two primary algorithmic paradigms — supervised machine learning regression and statistical time-series forecasting — alongside a custom composite scoring formula for ecological stress quantification.

2.1.1 Random Forest Regression

Random Forest (Breiman, 2001) is an ensemble learning method that constructs a multitude of decision trees at training time. Each tree is trained on a bootstrap sample of the training data, and at each split a random subset of features is considered, ensuring diversity among trees. The final prediction is the average of individual tree predictions.

For this system, the Random Forest Regressor is configured with `n_estimators = 100` trees and a fixed random seed for reproducibility. The target variable is `Pilgrim_Count` and the feature set consists of `Month` (integer 1–12), `Avg_Temperature_C`, and `Rainfall_mm`. The scikit-learn implementation is employed, with an 80/20 train-test split. Model performance is evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the Coefficient of Determination (R^2). Feature importance scores are extracted from the fitted ensemble to identify the relative contribution of each predictor.

2.1.2 ARIMA Time-Series Forecasting

The AutoRegressive Integrated Moving Average (ARIMA) model, parameterised as $ARIMA(p, d, q)$, captures temporal autocorrelation in pilgrim count time series. The parameter p (autoregressive order) captures dependency on past observations, d (degree of differencing) ensures stationarity, and q (moving average order) models the dependence on past forecast errors. The statsmodels library is used for ARIMA fitting. The optimal order is selected based on domain knowledge and Akaike Information Criterion (AIC) minimisation. The model is trained on the historical pilgrim count series and used to forecast future monthly counts.

2.1.3 Ecosystem Stress Index (ESI)

The ESI is a weighted composite index defined as:

$$ESI = w_1 \times (Pilgrim_Count / Carrying_Capacity) + w_2 \times |Temp - Temp_Mean| / Temp_Std + w_3 \times |Rainfall - Rain_Mean| / Rain_Std$$

where $w_1 = 0.5$, $w_2 = 0.3$, and $w_3 = 0.2$ represent domain-expert-assigned weights. All three sub-indices are min-max normalised prior to aggregation. An ESI value below 0.4 corresponds to

Green (Low Stress), between 0.4 and 0.7 corresponds to Yellow (Moderate Stress), and 0.7 or above corresponds to Red (Critical Stress).

2.2 Characteristic of Data

The system operates on two structured tabular datasets sourced from government tourism statistics and meteorological records covering the Char Dham pilgrimage region.

2.2.1 Tourist Footfall Dataset

Column	Data Type	Description	Source
Year	Integer	Calendar year of observation	UTDB
Month	Integer (1–12)	Month of observation	UTDB
Shrine	String	Name of pilgrimage site	UTDB
District	String	Administrative district	UTDB
Pilgrim_Count	Integer	Monthly visitor count	Sacred Yatra
Carrying_Capacity	Integer	Max sustainable pilgrim count	NGT Directive
Latitude	Float	Geographical latitude (WGS84)	Survey of India
Longitude	Float	Geographical longitude (WGS84)	Survey of India

Table 2.1: Tourist Footfall Dataset Schema

2.2.2 Climate Dataset

Column	Data Type	Description	Source
Year	Integer	Calendar year	IMD
Month	Integer (1–12)	Month of observation	IMD
Shrine	String	Pilgrimage site name	IMD
District	String	Administrative district	IMD
Avg_Temperature_C	Float	Monthly mean temperature (°C)	IMD / Satellite
Rainfall_mm	Float	Monthly total rainfall (mm)	IMD / Satellite

Table 2.2: Climate Dataset Schema

Data Integration: Both datasets are merged on the composite key (Year, Month, Shrine, District) using a Pandas left-join operation. Missing values arising from unmatched keys are handled through median imputation for numerical columns and mode imputation for categorical columns. Outliers exceeding ± 3 standard deviations from the column mean are capped. The merged dataset is further enriched by deriving a Date column (datetime type) from Year and Month to enable time-series indexing. The final merged dataset spans multiple years and covers all four shrines with monthly granularity, yielding a balanced panel dataset suitable for both supervised learning and time-series modelling.

2.3 SWOT Analysis

STRENGTHS	WEAKNESSES
- Integration of two heterogeneous datasets (football + climate) provides richer predictive power than single-source approaches. - Dual modelling strategy (Random Forest + ARIMA) enables both feature-driven and temporal pattern capture. - SHAP explainability increases stakeholder trust and regulatory acceptance. - Streamlit-based deployment requires no dedicated server or database infrastructure, reducing total cost of ownership. - ESI composite index provides a single, interpretable metric for ecosystem health monitoring.	- Dependence on historically aggregated monthly data limits granularity; daily or hourly real-time data would improve alerting responsiveness. - ESI weight coefficients are heuristically assigned; lack of empirical validation may reduce ecological accuracy. - ARIMA assumes stationary and linearly correlated series, which may underperform during anomalous years (e.g., pandemic-affected 2020–21). - Mock weather simulation does not constitute actual real-time data integration.
OPPORTUNITIES	THREATS
- Expansion to other pilgrimage circuits (Panch Kedar, Nanda Devi) using the same pipeline. - Integration with live IoT sensors and CCTV crowd analytics to replace mock weather data.	- Data quality and completeness are contingent on government agency cooperation; incomplete records could degrade model accuracy. - Climate change may shift historical statistical baselines, reducing forecasting reliability over long horizons.

• API-based connection to the Uttarakhand State Data Centre for automated dataset refresh. • Potential commercialisation as a SaaS offering for state tourism boards across India. • Addition of LSTM/GRU deep learning models to improve long-horizon forecasting accuracy.	• Regulatory policies imposing entry restrictions could render predictive models temporarily irrelevant. • Cybersecurity threats to the dashboard, particularly if extended to a public web deployment.
--	--

Table 2.3: SWOT Analysis of the AI Tourism Pressure Dashboard

2.4 Project Features

The system provides the following major functional features, each corresponding to a dedicated module within the Streamlit dashboard:

F#	Feature Name	Description
F1	Data Integration & Processing	Automated loading, merging, and cleaning of footfall and climate datasets using Pandas.
F2	Exploratory Data Analysis	Interactive line charts, scatter plots, seasonality heatmaps, and correlation matrices rendered via Plotly.
F3	ESI Computation	Weighted composite index combining pilgrim density, temperature deviation, and rainfall deviation per shrine.
F4	Random Forest Prediction	Supervised regression model predicting monthly pilgrim counts using climate features.
F5	ARIMA Forecasting	Time-series forecasting of pilgrim counts from historical temporal data.
F6	Model Evaluation	RMSE, MAE, R^2 metrics and residual/feature importance visualisations for model assessment.
F7	Early Warning Alert System	Automated Green/Yellow/Red alert generation based on predicted ESI values.
F8	AI Recommendation Engine	Contextual operational recommendations (e.g., controlled entry, permit restrictions) per shrine.
F9	Geospatial Map	Interactive map displaying shrine locations, pilgrim counts, carrying capacities, and ESI status.
F10	SHAP Explainability	SHAP value computation and visualisation for interpretable Random Forest predictions.
F11	Mock Real-Time Weather	Simulated current weather values based on historical dataset distribution for demonstration.

Table 2.4: Project Feature Summary

2.5 User Classes and Characteristics

User Class	Technical Proficiency	Primary Interaction
Tourism Authorities	Low to Medium	Monitor ESI alerts, configure carrying capacity thresholds, review AI recommendations, export reports.
Environmental Agencies	Medium	Inspect ESI trends over time, analyse geospatial maps, download stress metric reports.
Research Scholars	High	Explore correlation matrices, SHAP values, model evaluation metrics, and raw dataset statistics.
Policy Makers	Low	View summary dashboards, receive early warning alerts, and consult AI recommendations for policy formulation.

Table 2.5: User Classes and Characteristics

2.6 Design and Implementation Constraints

- **Programming Language Constraint:** The entire system is implemented in Python 3.9 or above. No component may be developed in another language without explicit approval.
- **Framework Constraint:** The user interface must be delivered as a Streamlit web application. Alternative frameworks (e.g., Django, Flask, Dash) are not permitted in the current scope.
- **Library Compatibility:** All Python libraries (Pandas, NumPy, Scikit-learn, Statsmodels, Plotly, SHAP) must be compatible with the versions specified in requirements.txt to ensure reproducibility.
- **Data Format Constraint:** Input datasets must be provided in CSV format with the exact column schema described in Section 2.2. Deviation in column naming or data type will cause a runtime error.
- **Memory Constraint:** The system is designed to operate on a standard laptop with a minimum of 8 GB RAM. Datasets exceeding 500,000 rows may cause memory pressure and require chunked loading.
- **No Real-Time Data API:** The current version does not integrate live APIs. Weather simulation is mock-based and not intended for operational forecasting.
- **No Database Backend:** All data persistence is file-based (CSV). The system does not use a relational database management system (RDBMS) or NoSQL store.
- **No Authentication Layer:** The dashboard does not implement user authentication or role-based access control. This constraint must be addressed in future production deployments.
- **Display Resolution:** The Streamlit UI is optimised for a minimum screen resolution of 1280×720 pixels. Smaller resolutions may result in layout degradation.

- Internet Connectivity: An active internet connection is required at launch for Streamlit's CDN-based asset loading. Subsequent operations are local.

2.7 Design Diagrams

2.7.1 System Architecture Diagram

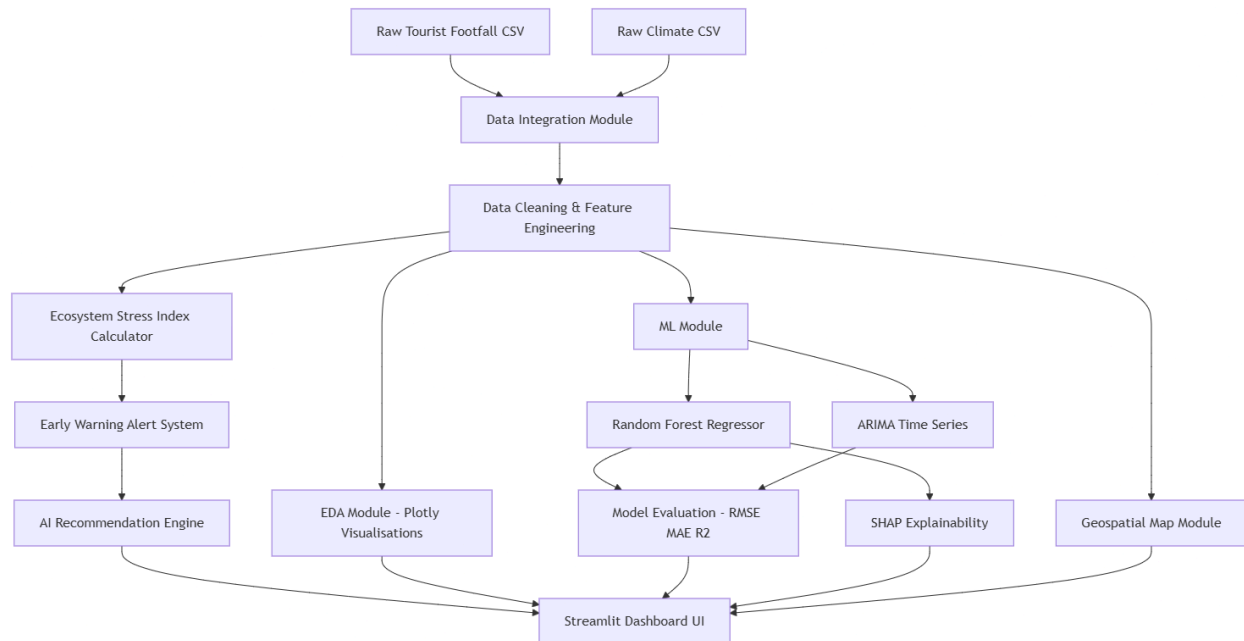


Figure 2.1: System Architecture Diagram

2.7.2 Use Case Diagram

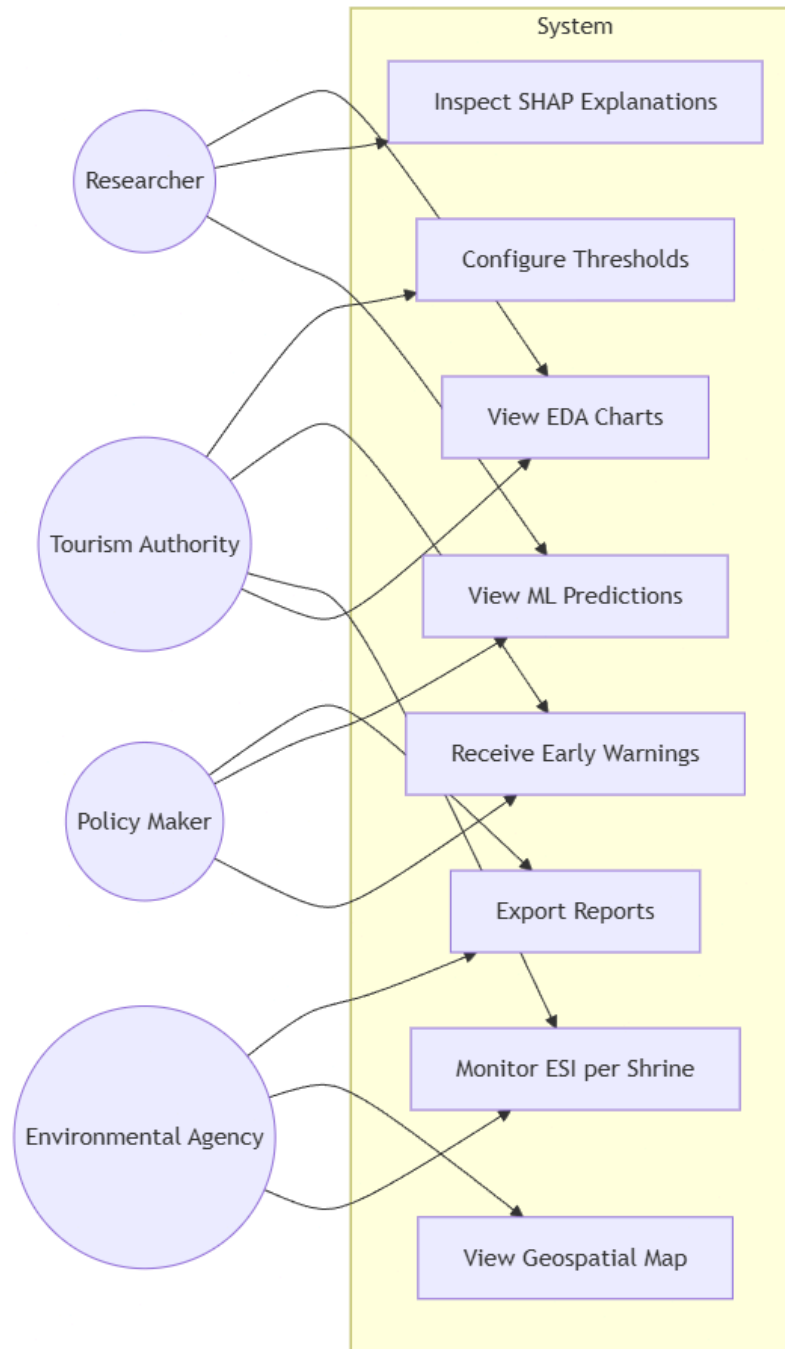


Figure 2.2: Use Case Diagram

2.7.3 Class Diagram

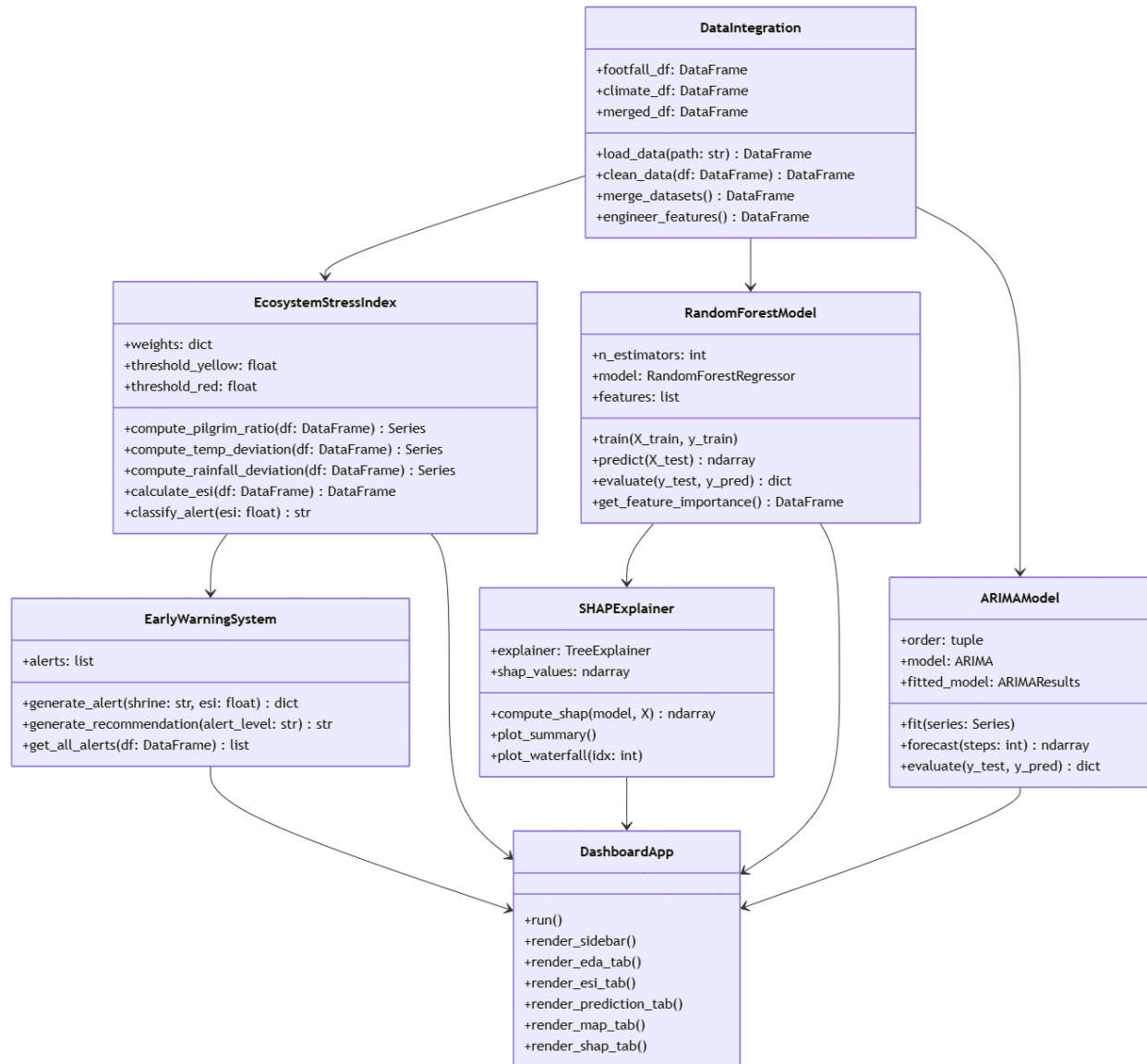


Figure 2.3: Class Diagram

2.7.4 Activity Diagram

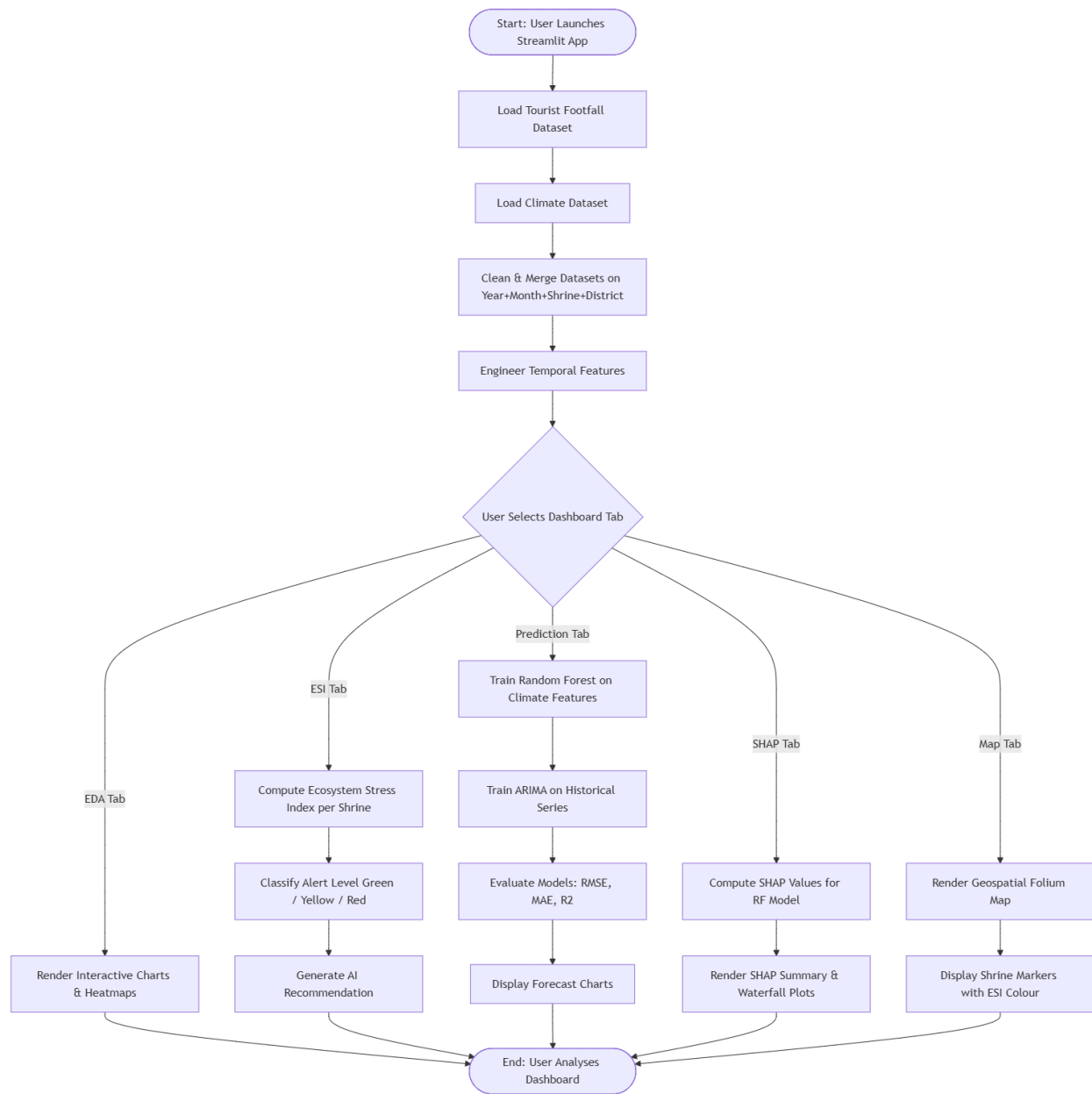


Figure 2.4: Activity Diagram

2.7.5 Sequence Diagram

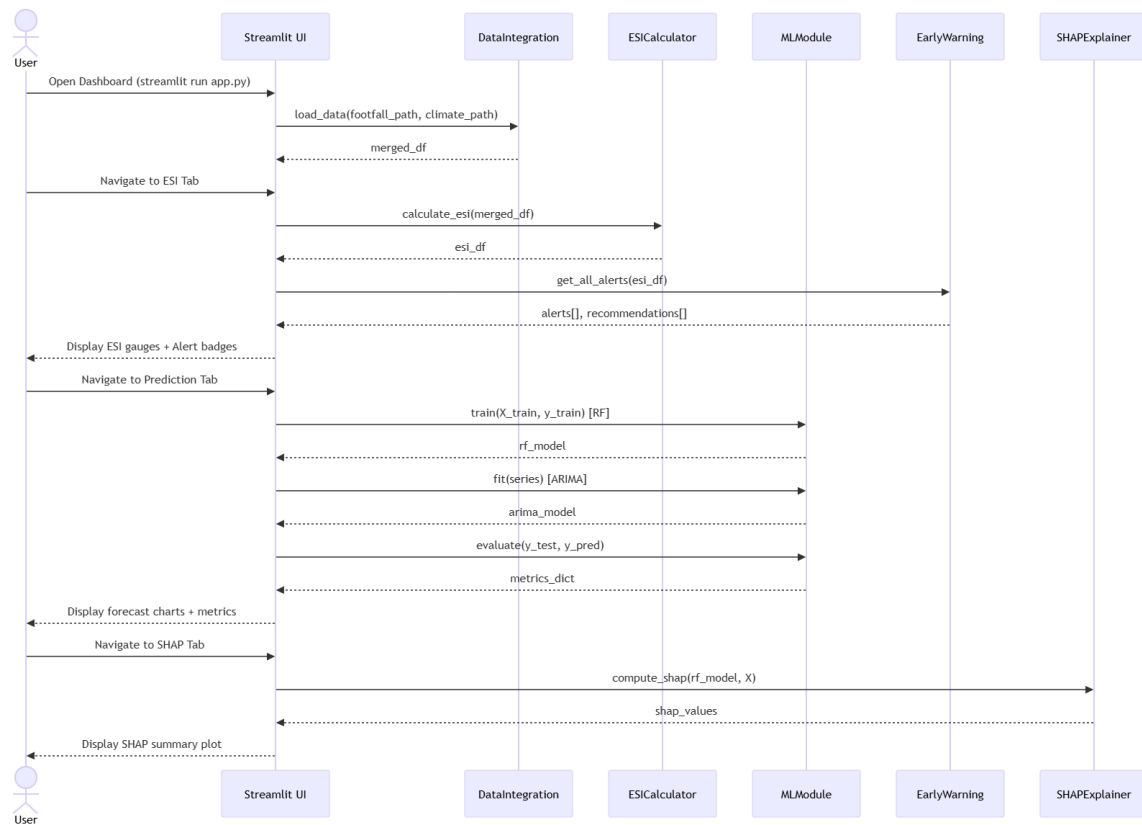


Figure 2.5: Sequence Diagram

2.7.6 Data Flow Diagram – Level 0 (Context Diagram)

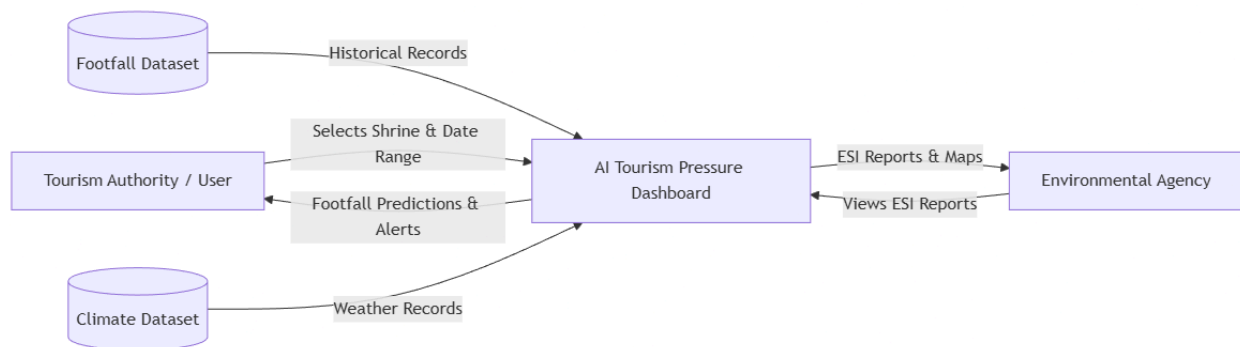


Figure 2.6: DFD Level 0 – Context Diagram

2.7.7 Data Flow Diagram – Level 1

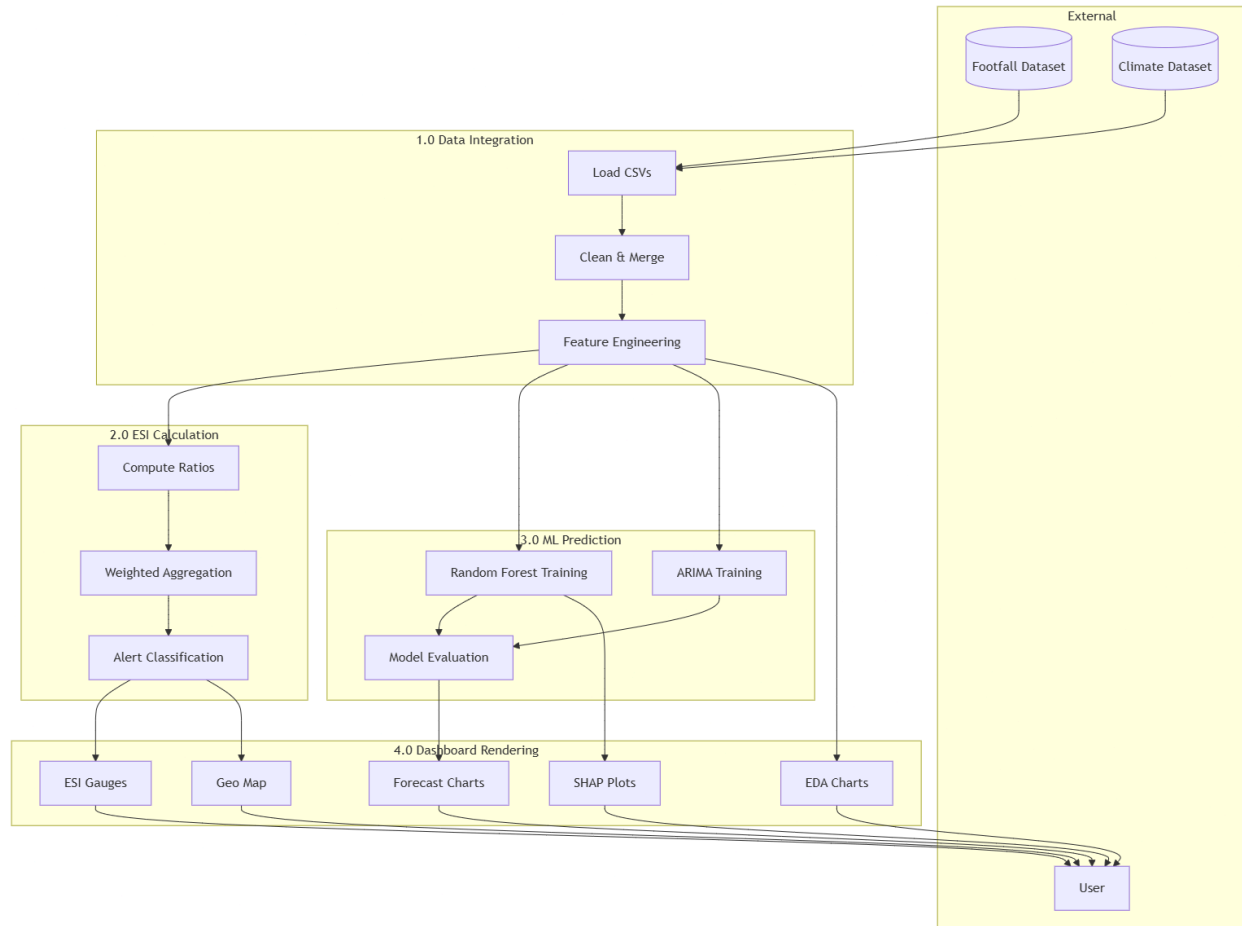


Figure 2.7: DFD Level 1

2.7.8 State Diagram

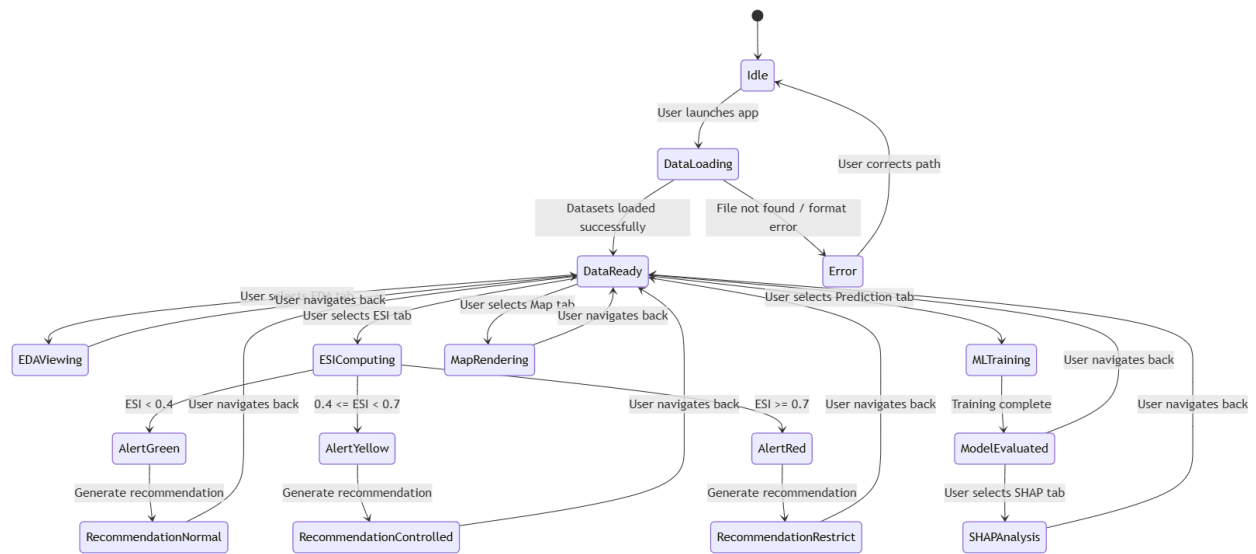


Figure 2.8: State Transition Diagram

2.7.9 Deployment Diagram

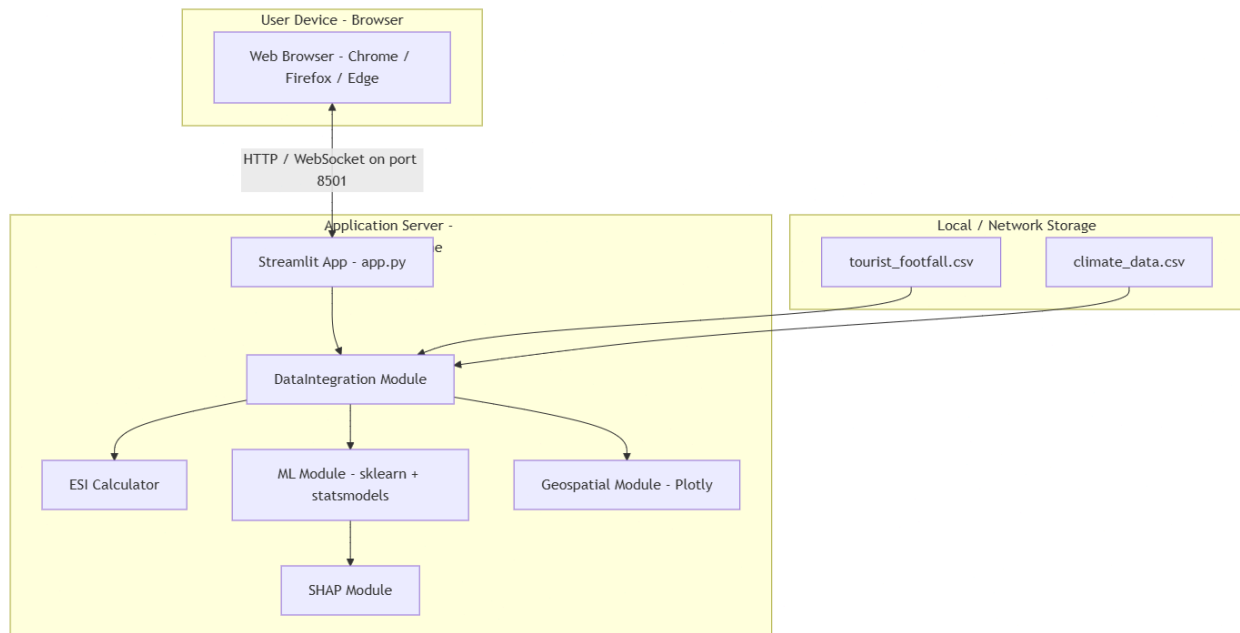


Figure 2.9: Deployment Diagram

2.8 Assumptions and Dependencies

Assumptions:

- It is assumed that the Tourist Footfall and Climate datasets provided are representative, accurate, and free of systematic biases introduced during collection.
- Shrine-level carrying capacities are assumed to remain constant as defined in the NGT directive and do not change dynamically within the system's operational period.
- ESI weight coefficients (0.5, 0.3, 0.2) are assumed to be a reasonable initial approximation pending empirical calibration by ecological domain experts.
- The historical dataset is assumed to exhibit sufficient seasonal and temporal variation to train a meaningful ARIMA model without requiring external regressors.
- Users are assumed to access the dashboard via a modern web browser (Chrome, Firefox, or Edge) with JavaScript enabled.
- The local Python environment is assumed to have all required packages installed prior to application launch.

Dependencies:

- Python 3.9+: Core runtime environment for all modules.
- Streamlit ($\geq 1.28.0$): Web application framework and UI rendering engine.
- Pandas ($\geq 1.5.0$): Data loading, manipulation, and merging.
- NumPy ($\geq 1.23.0$): Numerical operations and array processing.
- Scikit-learn ($\geq 1.2.0$): Random Forest Regressor, train-test split, and evaluation metrics.
- Statsmodels ($\geq 0.13.0$): ARIMA model fitting and forecasting.
- Plotly ($\geq 5.14.0$): Interactive charting and geospatial map rendering.
- SHAP ($\geq 0.41.0$): SHapley Additive exPlanations for model interpretability.
- Tourist Footfall Dataset (CSV): Primary external data dependency for pilgrim and capacity records.
- Climate Dataset (CSV): Primary external data dependency for meteorological records.

3 SYSTEM REQUIREMENTS

3.1 User Interface

The user interface is implemented entirely within the Streamlit framework, which renders an interactive, reactive web application accessible via a standard browser at the default address <http://localhost:8501>. The UI is structured as a multi-tab single-page application with the following components:

UI Component	Description
Sidebar Navigation Panel	Houses shrine selector, year/month range sliders, and global filter controls. Persists across all tabs.
Tab 1 – Data Overview	Displays merged dataset table, data shape, null-value summary, and descriptive statistics.
Tab 2 – EDA	Interactive Plotly charts: pilgrim trend line, climate–tourism scatter, seasonality heatmap, correlation matrix.
Tab 3 – ESI & Alerts	Gauge charts per shrine, alert-level badges (Green/Yellow/Red), and dynamically generated AI recommendations.
Tab 4 – ML Predictions	RF prediction vs. actual chart, ARIMA forecast horizon chart, model evaluation metrics table.
Tab 5 – Geospatial Map	Plotly interactive scatter-geo map with colour-coded shrine markers, tooltips showing pilgrim count and ESI.
Tab 6 – SHAP Explainability	SHAP summary beeswarm plot and individual waterfall plots for selected predictions.
Mock Weather Panel	Sidebar sub-panel displaying simulated current weather (temperature and rainfall) per shrine.

Table 3.1: User Interface Components

All visualisations are rendered using Plotly Express and Plotly Graph Objects, embedded via `st.plotly_chart()` with the `use_container_width=True` parameter. The layout uses `st.columns()` for multi-column arrangements and `st.expander()` for collapsible detail panels. The colour scheme adheres to WCAG 2.1 AA contrast standards.

3.2 Software Interface

The following software interfaces define the inter-module communication within the application:

Interface	Protocol / Mechanism	Description
-----------	----------------------	-------------

DataIntegration → ESICalculator	Python function call, Pandas DataFrame	Merged DataFrame passed as argument to ESI computation function.
DataIntegration → MLModule	Python function call, NumPy arrays	Feature matrix X and target vector y extracted from merged DataFrame.
MLModule → SHAPExplainer	Python object reference	Fitted RandomForestRegressor and feature matrix passed to SHAP TreeExplainer.
EarlyWarningSystem → DashboardApp	Python list/dict return	Alert dictionaries containing shrine name, ESI value, alert level, and recommendation string.
DashboardApp → Browser	HTTP/WebSocket (Streamlit protocol)	Streamlit serialises UI components to JSON and streams them via WebSocket to the browser.
Browser → DashboardApp	WebSocket events	User interactions (slider changes, tab clicks) trigger Streamlit re-runs via WebSocket callbacks.

Table 3.2: Software Interface Specifications

All intra-application interfaces are synchronous Python function calls operating within a single process. There are no inter-process or inter-service communication channels in the current architecture. Streamlit's reactive execution model re-runs the entire script on each user interaction, ensuring consistency between UI state and computed outputs.

3.3 Database Interface

The current system does not utilise a relational database management system (RDBMS) or a NoSQL data store. All data persistence and retrieval is accomplished through the local file system using CSV-formatted flat files. The Pandas library serves as the data access layer, providing `read_csv()` and `to_csv()` interfaces for loading and exporting data respectively.

File Name	Format	Access Mode	Usage
tourist_footfall.csv	CSV (UTF-8)	Read-Only	Primary dataset ingestion
climate_data.csv	CSV (UTF-8)	Read-Only	Primary dataset ingestion
merged_dataset.csv (optional)	CSV (UTF-8)	Read/Write	Cached merged output

Table 3.3: File-Based Data Interface

Future versions of this system should integrate a relational database (e.g., PostgreSQL) to support concurrent multi-user access, row-level security, transaction management, and automated data refresh pipelines. The SQLAlchemy ORM is recommended as the abstraction layer for database connectivity in such a scenario.

3.4 Protocols

The following communication and operational protocols govern the system's behaviour:

- HTTP/1.1 and WebSocket Protocol: Streamlit uses HTTP for initial page load and upgrades to a WebSocket connection (ws://) for all subsequent bi-directional communication between the browser client and the Python server. All communication occurs on TCP port 8501 by default.
- File I/O Protocol: CSV files are read using Pandas `pd.read_csv()` with explicit `encoding='utf-8'` and `parse_dates` parameters. Date parsing follows ISO 8601 (YYYY-MM-DD) convention.
- Data Security Protocol: In the current local deployment, no transport-layer encryption (TLS/SSL) is applied. For public-facing deployments, HTTPS (TLS 1.2 or higher) must be enforced using a reverse proxy (e.g., Nginx) with a valid SSL certificate.
- Session State Protocol: Streamlit's `st.session_state` dictionary is used to persist model objects, trained parameters, and user filter selections across re-runs within a single browser session. Session state is not persisted across server restarts.
- Random Seed Protocol: All stochastic operations (Random Forest training, train-test split) use a fixed `random_state = 42` to ensure reproducibility across executions.
- Geospatial Coordinate System: All geographical coordinates use the WGS84 (EPSG:4326) coordinate reference system, consistent with standard web mapping conventions.

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

The following performance benchmarks are defined for the system under standard hardware conditions (Intel Core i5 or equivalent, 8 GB RAM, SSD storage):

Performance Criterion	Target	Notes
Application Cold Start Time	≤ 10 seconds	From streamlit run command to UI ready in browser.
Dataset Loading & Merging	≤ 5 seconds	For datasets up to 10,000 records.
ESI Computation	≤ 2 seconds	Vectorised Pandas operations; no loops.
Random Forest Training	≤ 15 seconds	100 estimators, 80% training split.
ARIMA Model Fitting	≤ 20 seconds	Depends on series length and (p,d,q) order.
SHAP Value Computation	≤ 30 seconds	For dataset up to 1,000 test samples.
Chart Rendering (Plotly)	≤ 3 seconds per chart	Browser-side WebGL rendering.
UI Re-render on Filter Change	≤ 2 seconds	Streamlit reactive re-run cycle.
System Uptime	$\geq 99\%$ (local deployment)	Dependent on host machine availability.

Table 4.1: Performance Requirements

4.2 Security Requirements

4.2.1 Authentication and Authorisation

In its current local deployment form, the system does not enforce user authentication. For production deployment scenarios, the following security measures are mandated:

- User authentication shall be implemented using OAuth 2.0 with JSON Web Tokens (JWT) or government-issued PKI credentials.
- Role-Based Access Control (RBAC) shall restrict access to configuration panels (e.g., threshold editing) to authorised Tourism Authority administrators only.
- Session tokens shall expire after 30 minutes of inactivity.

4.2.2 Data Protection

- Tourist footfall and climate datasets do not contain personally identifiable information (PII); however, all data files shall be stored with read-only permissions for the application process user.
- For cloud deployments, data at rest shall be encrypted using AES-256 encryption standard.
- Data in transit shall be protected by TLS 1.2 or higher using HTTPS.

4.2.3 Input Validation

- All user-supplied filter inputs (slider ranges, dropdown selections) are validated within Streamlit's widget API; custom validation logic shall reject out-of-range or type-mismatched inputs.
- CSV upload functionality (if added in future) shall include file type, size, and schema validation before processing.

4.2.4 Audit and Logging

- All critical operations (data load events, model training initiations, alert generation) shall be logged to a timestamped application log file (app.log) for audit trail purposes.

4.3 Software Quality Attributes

Quality Attribute	Implementation Strategy
Adaptability	Modular code architecture with separate Python modules for DataIntegration, ESICalculator, MLModule, and EarlyWarningSystem. New data sources or models can be added without modifying the dashboard layer.
Availability	The system is designed for continuous local operation. Streamlit's auto-reload on code changes supports development-phase availability. Production deployments should use a process manager (e.g., systemd, pm2) to auto-restart on failure.
Correctness	Model predictions are validated against a held-out test set using RMSE, MAE, and R ² metrics. ESI calculations are unit-tested for boundary conditions (ESI = 0 when Pilgrim_Count = 0, ESI = 1 at maximum stress).
Flexibility	ESI weight coefficients and alert thresholds are configurable via a settings dictionary without code modification. Additional shrines can be incorporated by extending the input dataset.
Interoperability	CSV input format ensures compatibility with any spreadsheet or database export tool. Plotly charts support PNG/SVG export for integration with external reporting systems.
Maintainability	Code adheres to PEP 8 style guidelines, includes inline docstrings for all functions, and uses version-pinned requirements.txt to prevent dependency drift.

Portability	The application runs on any OS supporting Python 3.9+ (Windows 10+, Ubuntu 20.04+, macOS 12+). Docker containerisation is recommended for environment-agnostic deployment.
Reliability	Graceful error handling is implemented for missing files, malformed CSV data, and model convergence failures. Informative error messages are displayed via Streamlit's <code>st.error()</code> widget.
Reusability	Core modules (<code>DataIntegration</code> , <code>ESICalculator</code> , <code>MLModule</code>) are designed as standalone Python classes importable into external scripts or Jupyter notebooks for research purposes.
Robustness	The system handles edge cases including zero pilgrim counts (division-by-zero guards in ESI), missing monthly records (median imputation), and insufficient data for ARIMA (minimum 24 observations check).
Testability	All modules expose well-defined input/output contracts amenable to unit testing using <code>pytest</code> . Fixtures for mock datasets are included in the <code>test/</code> directory.
Usability	The Streamlit interface requires no installation by end users beyond browser access. Contextual tooltips, chart legends, and sidebar help text reduce the learning curve. Alert colour coding follows international traffic-light conventions.

Table 4.2: Software Quality Attributes

5 OTHER REQUIREMENTS

The following requirements do not fall within the functional or non-functional categories defined in Sections 3 and 4 but are nonetheless binding for the successful delivery and evaluation of this project.

5.1 Legal and Regulatory Compliance

- The system must comply with the National Green Tribunal (NGT) directives on carrying capacity limits for the Char Dham shrines. Carrying capacity values embedded in the dataset must reflect the most recently issued NGT order.
- Data collected from government portals must be used in accordance with the National Data Sharing and Accessibility Policy (NDSAP) of India.
- The project must not publish or redistribute any raw government dataset without prior written consent from the issuing authority.

5.2 Ethical Requirements

- The AI recommendation engine must not generate suggestions that could be construed as discriminatory on the basis of caste, religion, or pilgrim denomination. Recommendations are based solely on quantitative ESI values and pilgrim counts.
- SHAP explanations must be accessible within the dashboard to ensure algorithmic transparency for all stakeholder groups.
- The system must include a clear disclaimer that model predictions are decision-support tools and should not substitute expert human judgement in emergency situations.

5.3 Documentation Requirements

- A comprehensive User Manual must be delivered alongside the software, detailing installation, dataset preparation, and dashboard operation procedures.
- Inline code documentation (docstrings) must cover all public methods and classes, conforming to the NumPy docstring convention.
- A requirements.txt file with pinned dependency versions must be maintained and updated with each release.

5.4 Internationalisation and Localisation

- The current version of the dashboard operates exclusively in the English language. Future versions should support Hindi-language UI localisation to increase accessibility for regional tourism authority users.
- All numerical outputs use the Indian number formatting convention (lakh/crore) in display labels, while internal computations use raw integers.

5.5 Sustainability and Environmental Requirements

- The computational workload of the system (particularly ARIMA fitting and SHAP computation) should be benchmarked for energy consumption. Batch training modes that defer computation to off-peak hours are preferred for cloud deployments.

APPENDIX A: GLOSSARY

Term / Acronym	Definition
ARIMA	AutoRegressive Integrated Moving Average – a statistical model for time-series forecasting.
AIC	Akaike Information Criterion – a measure used for model selection; lower values indicate better fit.
Carrying Capacity	The maximum number of visitors a shrine or site can sustain without causing unacceptable ecological degradation.
Char Dham	The four sacred Hindu pilgrimage sites in Uttarakhand: Kedarnath, Badrinath, Gangotri, and Yamunotri.
CSV	Comma-Separated Values – a plain-text file format for tabular data.
EDA	Exploratory Data Analysis – statistical and graphical methods used to summarise and discover patterns in a dataset.
ESI	Ecosystem Stress Index – a composite metric quantifying environmental pressure on a shrine site.
IMD	India Meteorological Department – the national weather agency providing climate data.
JWT	JSON Web Token – a compact, URL-safe token format used for authentication and authorisation.
MAE	Mean Absolute Error – the average of absolute differences between predicted and actual values.
ML	Machine Learning – a subset of artificial intelligence in which algorithms learn from data.
NGT	National Green Tribunal – an Indian judicial body for environmental cases.
NDSAP	National Data Sharing and Accessibility Policy – India's framework for open government data.
R^2	Coefficient of Determination – a statistical measure of how well the model explains variance in the target variable.
RBAC	Role-Based Access Control – a method of restricting system access based on user roles.
RF	Random Forest – an ensemble machine learning algorithm based on multiple decision trees.
RMSE	Root Mean Squared Error – the square root of the mean of squared prediction errors.
SRS	Software Requirements Specification – a document describing the intended behaviour and constraints of a software system.
SHAP	SHapley Additive exPlanations – a game-theoretic framework for explaining the output of machine learning models.

TLS	Transport Layer Security – a cryptographic protocol for securing communications over a network.
UTDB	Uttarakhand Tourism Development Board – the state body responsible for tourism planning and data collection.
WGS84	World Geodetic System 1984 – the standard coordinate reference system used in GPS and web mapping.

APPENDIX B: ANALYSIS MODEL

This appendix presents supplementary analytical models that provide additional context for the system's design and predictive methodology.

B.1 ESI Computation Model

The Ecosystem Stress Index is derived through a three-stage normalisation and aggregation process:

Stage 1 – Sub-index Calculation: $\text{Pilgrim Ratio} = \text{Pilgrim_Count} / \text{Carrying_Capacity}$
 $\text{Temp Deviation} = |\text{Avg_Temperature_C} - \text{Temp_Mean}| / \text{Temp_Std}$
 $\text{Rain Deviation} = |\text{Rainfall_mm} - \text{Rain_Mean}| / \text{Rain_Std}$

Stage 2 – Min-Max Normalisation (per shrine): $\text{Normalised_x} = (\text{x} - \text{x_min}) / (\text{x_max} - \text{x_min}) \in [0, 1]$

Stage 3 – Weighted Aggregation: $\text{ESI} = 0.5 \times \text{Norm}(\text{PilgrimRatio}) + 0.3 \times \text{Norm}(\text{TempDev}) + 0.2 \times \text{Norm}(\text{RainDev})$

Alert Classification: $\text{ESI} < 0.40 \rightarrow \text{GREEN}$ (Low Stress) $\rightarrow \text{Normal Monitoring}$
 $0.40 \leq \text{ESI} < 0.70 \rightarrow \text{YELLOW}$ (Moderate) $\rightarrow \text{Controlled Entry}$
 $\text{ESI} \geq 0.70 \rightarrow \text{RED}$ (Critical Stress) $\rightarrow \text{Permit Restrictions}$

B.2 Random Forest Feature Importance Ranking

Post-training feature importance scores from the Random Forest model, ordered by mean decrease in impurity (MDI), are expected to rank the predictors as follows based on domain knowledge:

Rank	Feature	Rationale
1	Month	Pilgrim footfall is highly seasonal; Char Dham opens only between May and November.
2	Avg_Temperature_C	Temperature strongly correlates with pilgrimage season; extreme cold or heat deters visitors.
3	Rainfall_mm	Heavy monsoon rainfall reduces accessibility; moderate rainfall has limited influence.

Table B.1: Expected Random Forest Feature Importance Ranking

B.3 ARIMA Model Order Selection

Stationarity of the monthly pilgrim count series is assessed using the Augmented Dickey-Fuller (ADF) test. If the null hypothesis of a unit root is not rejected ($p > 0.05$), first-order differencing ($d = 1$) is applied. The partial autocorrelation function (PACF) is examined to determine the

autoregressive order p , and the autocorrelation function (ACF) guides selection of the moving average order q . The expected optimal order for the Char Dham pilgrim series is ARIMA(2, 1, 1), subject to empirical validation.

APPENDIX C: ISSUES LIST

This appendix documents open requirements issues, known limitations, and deferred enhancements identified during the development and review of this SRS. Each issue is assigned a unique identifier, priority, and current status.

ID	Issue Description	Priority	Status	Resolution Plan
C-01	ESI weight coefficients are heuristically assigned and lack empirical ecological validation.	High	Open	Engage ecological domain experts for calibration in v2.0.
C-02	Mock weather data is not derived from live meteorological feeds.	High	Deferred	Integrate IMD Weather API in v2.0.
C-03	No user authentication or role-based access control is implemented.	High	Deferred	Implement OAuth 2.0 + RBAC in production release.
C-04	ARIMA model may underperform for anomalous years (e.g., COVID-19 2020–21).	Medium	Open	Explore outlier-robust forecasting (SARIMA, Prophet) in future work.
C-05	Dataset refresh is entirely manual; no automated data pipeline exists.	Medium	Deferred	Develop ETL pipeline with scheduled cron job for v2.0.
C-06	Dashboard does not support Hindi-language localisation.	Low	Deferred	Implement i18n framework in v3.0.
C-07	No export functionality for PDF or Excel reports.	Low	Open	Integrate pdfkit or openpyxl export in v2.0.
C-08	System has not been load-tested for concurrent multi-user access.	Medium	Open	Conduct load testing with Locust tool before production deployment.
C-09	SHAP computation is slow (>30 sec) for large test sets.	Low	Open	Implement background threading or SHAP sampling for larger datasets.
C-10	The system does not support dynamic addition of new shrines from the UI.	Low	Deferred	Add shrine management interface in v2.0.

Table C.1: Open Issues and Deferred Enhancements